Lab 04 – (Week of September 16th)

Activity 1: Activity 1: Matching Source Code with Output (See Separate Sheet)

All of the following remaining activities of this lab are to be completed in teams of two (or three, but only as necessary), and each team is limited to the use of one computer.

Activity #2: Extending `SquarePerimeters` into `RectanglePerimeters`

Consider the given program, `SquarePerimeters.java`, that when run prompts the user to provide an interval that defines the measures of a series of squares. The program proceeds to calculate and print the corresponding perimeters of each square defined by the interval. Thus, in the example shown to the right, a square of length 1 has a perimeter of 4, a square of length 4 has a perimeter of 16, etc.

```
Square Perimeters ...
Enter the start value >:1
Enter the stop value >:10
Enter the increment >:3
      ||
------++------+
    1 ||    4 |
    4 ||   16 |
    7 ||   28 |
   10 ||   40 |
------++------+
```

For this activity you are to rename `SquarePerimeters.java`, as `RectanglePerimeters.java` and then modify it so that instead of computing and printing just the perimeters of squares it computes and prints the perimeters of those rectangles defined by the given interval; varying both the length and width accordingly.

```
Rectangle Perimeters ...
Enter the start value >:1
Enter the stop value >:10
Enter the increment >:3
      ||    1 |    4 |    7 |   10 |
------++------+------+------+------+
    1 ||    4 |   10 |   16 |   22 |
    4 ||   10 |   16 |   22 |   28 |
    7 ||   16 |   22 |   28 |   34 |
   10 ||   22 |   28 |   34 |   40 |
------++------+------+------+------+
```

Consider the second example that depicts the same interval used for both the length and width of a series of rectangles. In this example, notice that a rectangle of length 4 and width 1 has a perimeter of 10, a rectangle of length 7 and width 10 has a perimeter of 34. Likewise, note that those rectangles that are squares correspond to the diagonal of the tabular output (i.e. 4, 16, 28 and 40).

Here is a second example execution for your consideration.

```
Rectangle Perimeters ...
Enter the start value >:1
Enter the stop value >:32
Enter the increment >:3
      ||    1 |    4 |    7 |   10 |   13 |   16 |   19 |   22 |   25 |   28 |   31 |
------++------+------+------+------+------+------+------+------+------+------+------+
    1 ||    4 |   10 |   16 |   22 |   28 |   34 |   40 |   46 |   52 |   58 |   64 |
    4 ||   10 |   16 |   22 |   28 |   34 |   40 |   46 |   52 |   58 |   64 |   70 |
    7 ||   16 |   22 |   28 |   34 |   40 |   46 |   52 |   58 |   64 |   70 |   76 |
   10 ||   22 |   28 |   34 |   40 |   46 |   52 |   58 |   64 |   70 |   76 |   82 |
   13 ||   28 |   34 |   40 |   46 |   52 |   58 |   64 |   70 |   76 |   82 |   88 |
   16 ||   34 |   40 |   46 |   52 |   58 |   64 |   70 |   76 |   82 |   88 |   94 |
   19 ||   40 |   46 |   52 |   58 |   64 |   70 |   76 |   82 |   88 |   94 |  100 |
   22 ||   46 |   52 |   58 |   64 |   70 |   76 |   82 |   88 |   94 |  100 |  106 |
   25 ||   52 |   58 |   64 |   70 |   76 |   82 |   88 |   94 |  100 |  106 |  112 |
   28 ||   58 |   64 |   70 |   76 |   82 |   88 |   94 |  100 |  106 |  112 |  118 |
   31 ||   64 |   70 |   76 |   82 |   88 |   94 |  100 |  106 |  112 |  118 |  124 |
------++------+------+------+------+------+------+------+------+------+------+------+
```

When studying the given source program for `SquarePerimeters.java` you are encouraged to consider the three places marked with matching `//<<<<` and `//>>>>` comments. These comments have been carefully placed to draw your attention to those specific places where your modifications should be made. Essentially you will need to incorporate `for` loops in each of these places so as to extend the behavior of the program to the additional dimension required for the extension from squares to rectangles.

You are reminded of the merit of working incrementally and are thus strongly encouraged to accomplish the revisions one at a time, probably tackling the one in the `printTableBody` method first.

For this activity you are to submit your finished program, named `RectanglePerimeters.java`, to the CMPS 134L CWS for the section you are working in. However, before you do this you must both **execute it for your Lab Instructor** (to demonstrate that it produces the correct result) **and explain and discuss each of the modifications** with the Lab Instructor. Only when the Lab Instructor is satisfied are you to submit the file.

Be sure to modify the comment in the file to specify the full names of all team members.

Activity #3: Modifications to `Windows.java` involving constants and Additional Methods

Consider the given program, `Windows.java`, which when run produces the output shown to the right. This output first depicts a single panel window, known as 1x1 (i.e. one by one), then a two panel window with the panels running across, then a two panel window with the panels running down, then finally a four panel window with two panels across and two panels down.

```
Windows...
Window 1x1
+---+
|   |
|   |
|   |
+---+
Window 1x2
+---++---+
|   ||   |
|   ||   |
|   ||   |
+---++---+
Window 2x1
+---+
|   |
|   |
|   |
+---+
+---+
|   |
|   |
|   |
+---+
Window 2x2
+---++---+
|   ||   |
|   ||   |
|   ||   |
+---++---+
+---++---+
|   ||   |
|   ||   |
|   ||   |
+---++---+
```

Note that this program makes use of constants that express the height and width of the panel and for each of the specific character values used in the depiction.

For this activity you are to do each of the following sub-activities, in the order given. You are encouraged to show and explain the accomplishment of each sub-activity to your lab instructor before moving on to the next sub-activity.

a. Rename `Windows` as `WindowsV2`, then modify it so that the panels produced utilize panes that are 5 wide by 2 high.

b. Rename `WindowsV2` as `WindowsV3`, then modify it so that it produces panels that are of some reasonable width and height of your own choosing.

c. Rename `WindowsV2` as `WindowsV4` then modify it so that "joints" in the frame are represented by the '`*`' character rather than the '`+`' character, and the panes themselves are represented by '`.`' rather than ' '.

d. Rename `Windows` as `WindowsV5`, then develop each of the additional methods `Window4x1`, `Window1x4`, and `Window4x4`. Of course, you will need to modify `main` to call these methods rather than the methods `Window1x1`, `Window1x2`, `Window2x1`, and `Window2x2` as given.

e. If you have not done so already then be sure to show and explain your work on the earlier sub-activities to your lab instructor before moving on to this final sub-activity.

   Consider the given program, `WindowsFinal.java`, which presents two methods that need to be completed before it can run completely and properly. This program presents a single method, `WindowDxA`, that is designed to have the ability to depict any rectangular arrangement of panels as specified by the values of the two static variables, `numberDown`, and `numberAcross`.

For this activity you are to submit each of the following finished programs to the CMPS 134L CWS for the section you are working in. However, before you do this you must have both **executed each for your Lab Instructor** (to demonstrate that it produces the correct result) **and explain and discuss each** with the Lab Instructor. Only when the Lab Instructor is satisfied are you to submit the files.

- `WindowsV2.java`
- `WindowsV4.java`
- `WindowsV5.java`
- `WindowsFinal.java`

As always, be sure to provide comments in the files to specify the full names of all team members.